

PRACTICAL 3

Program1:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }

    return 0;
}
```

Output:

```
Parent Process
Parent PID : 497
Child PID : 498

Child Process
Child PID : 498
Parent PID : 497
```

Program2:

```
GNU nano 2.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process ID (PID)      : %d\n", getpid());
    printf("Parent Process ID(PPID) : %d\n", getppid());

    return 0;
}
```

Output:

```
Process ID (PID)      : 583
Parent Process ID(PPID) : 299
```

Program3:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        printf("Child is Running...\n");

        sleep(5);

        printf("Child Terminated.\n");
        exit(0);
    }
    else if (pid > 0)
    {
        printf("\nParent Process\n");
        printf("PID : %d\n", getpid());

        printf("Parent is Waiting for Child...\n");

        wait(NULL);

        printf("Child Finished.\n");
        printf("Parent Terminated.\n");
    }
    else
    {
        printf("Fork Failed!\n");
    }

    return 0;
}

```

Output:

```
nishanth@LAPTOP-6Q3ON07H:~/forklab$ gcc program3.c -o program3
nishanth@LAPTOP-6Q3ON07H:~/forklab$ ./program3
```

Parent Process

PID : 524

Parent is Waiting for Child...

Child Process

PID : 525

PPID : 524

Child is Running...

Child Terminated.

Child Finished.

Parent Terminated.